

Code Optimization in Compiler Design: Interview Guide

Code optimization is the compiler phase that improves generated code without changing program meaning. In interviews, optimization questions test whether you understand programs as data: expressions, variables, control flow, loops, calls, and runtime behavior.

Note: The correct spelling is "code optimization"; the file name follows the requested spelling: `code-optimization.md`.

1. Constant Folding

1. Overview

Definition: Constant folding evaluates constant expressions at compile time instead of runtime.

Example:

```
int x = 10 * 20;
```

can become:

```
int x = 200;
```

Why it matters: It removes unnecessary runtime computation, reduces instruction count, and often enables more optimizations.

Where used: C/C++ compilers, Java JIT compilers, JavaScript engines, database query optimizers, shader compilers, and mobile app compilers.

Why interviewers ask: It is one of the simplest optimizations, but it reveals whether you understand compile-time vs runtime evaluation and semantic preservation.

2. Core Idea

Intuition: If the compiler already knows all operands, it can calculate the answer immediately.

Analogy: Instead of asking a cashier to compute $50 + 20$ every time a customer buys the same bundle, the store prints the price as 70 .

Small example:

```
int a = 2 + 3 * 4;
```

Steps:

- 1. Parse expression: `2 + (3 * 4)`.
- 2. See `3` and `4` are constants.
- 3. Replace `3 * 4` with `12`.
- 4. Replace `2 + 12` with `14`.

Optimized:

```
int a = 14;
```

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Arithmetic folding	Evaluate arithmetic constants	Fewer CPU instructions	<code>4 * 8 -> 32</code>	Operator precedence and overflow
Boolean folding	Simplify boolean constants	Removes branches	<code>true && x -> x</code>	Short-circuit semantics
String folding	Join constant strings	Saves runtime concatenation	<code>"Hello " + "CS"</code>	Language-dependent behavior
Type-aware folding	Respect data type rules	Avoids wrong results	<code>int</code> overflow vs <code>float</code> rounding	Undefined behavior in C/C++
Target-aware folding	Consider machine details	Correct code generation	floating-point precision	Cross-platform correctness

4. Real-World Example

A browser JavaScript engine may compile:

```
const timeout = 60 * 1000;
```

into:

```
const timeout = 60000;
```

This avoids repeated arithmetic during script execution and helps later optimizations such as inlining or branch removal.

5. Diagrams / Mental Models

```

Source expression
  |
  v
Are all operands compile-time constants?
  |
  yes v
Evaluate safely using language rules
  |
  v
Replace expression with literal

```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is constant folding?	Compile-time evaluation of constant expressions.	Compile time, same result, fewer instructions	Saying it changes variables
Is <code>2 + 3</code> folded?	Yes, usually to <code>5</code> .	Basic arithmetic folding	Ignoring precedence
Is <code>x + 0</code> constant folding?	Usually algebraic simplification, not pure folding unless <code>x</code> is known.	Difference from simplification	Calling every simplification folding
Does folding affect runtime?	It reduces runtime work.	Performance benefit	Saying it always changes complexity
Can floating-point folding be tricky?	Yes, because rounding, precision, exceptions, and target behavior matter.	IEEE rules	Treating floats like exact integers
Can folding cause bugs?	Only if compiler ignores language semantics.	Overflow, UB, side effects	Folding unsafe expressions blindly
Is <code>strlen("abc")</code> folding?	It can be constant evaluation or builtin optimization.	Builtin knowledge	Calling it only folding
Is folding machine independent?	Mostly, but target details matter for floats and widths.	Target-aware correctness	Assuming all targets same
Does folding happen before parsing?	No, after parsing and semantic analysis.	Compiler pipeline	Confusing lexer with optimizer
Why do interviewers ask it?	It is a basic example of preserving meaning while improving code.	Foundation concept	Only giving an example, no principle

7. Deep-Dive Questions

Question	Answer
Can <code>1 / 0</code> be folded?	Usually not into a normal value; the compiler must respect language rules, diagnostics, traps, or undefined behavior.
Why is <code>0 * x</code> not always safe to replace with <code>0</code> ?	If evaluating <code>x</code> has side effects or traps, removing it changes behavior.
How does folding enable dead-code elimination?	A folded condition like <code>if (false)</code> exposes unreachable code.
How does constant folding differ in JIT compilers?	JITs can fold values known at runtime specialization time, not just source compile time.
Why can signed integer overflow matter?	In C/C++, signed overflow is undefined, so optimizers may assume it does not happen.

8. Comparison Tables

Constant Folding	Constant Propagation
Evaluates expressions already made of constants	Replaces variables with known constant values
Example: <code>3 * 4 -> 12</code>	Example: <code>x = 12; y = x + 1 -> y = 13</code>
Local expression-level optimization	Needs information about definitions and control flow
Often enables propagation	Often enables folding

9. Common Mistakes

- Thinking folding can evaluate expressions with side effects.
- Ignoring integer overflow and floating-point rounding.
- Confusing constant folding with constant propagation.
- Assuming it only applies to arithmetic.

10. Edge Cases / Special Cases

- `volatile` reads must not be folded away.
- Floating-point NaN, signed zero, rounding modes, and exceptions can restrict folding.
- Language rules decide whether overflow is defined, undefined, or wrapping.
- Function calls can be folded only if they are known pure and deterministic.

11. How to Explain in Interview

"Constant folding is when the compiler evaluates expressions made only of constants at compile time, like replacing `2 * 3 + 1` with `7`, as long as the replacement follows the language's exact semantics."

12. Quick Revision Notes

- Key definition: compile-time evaluation of constant expressions.
- Must remember: preserve semantics.

- Common comparison: folding evaluates expressions; propagation replaces variables.
- Trap: side effects and floating-point behavior.

13. Practice Tasks

1. Fold all constant expressions in `int x = (5 + 3) * (10 - 6);`.
2. Decide whether `0 * f()` can be replaced with `0`.
3. Write a small expression-tree evaluator for integer constants.
4. Find folded constants in compiler output using `gcc -O2 -S`.

14. Final Cheat Sheet

Item	Answer
Core definition	Evaluate constant expressions during compilation
Why it matters	Saves runtime work and exposes more optimizations
Most asked question	Difference between folding and propagation
Common comparison	Folding vs propagation
One-line answer	"The compiler computes known constant expressions before the program runs."

2. Constant Propagation

1. Overview

Definition: Constant propagation replaces uses of a variable with a known constant value when the compiler can prove the variable has that value.

```
int a = 10;
int b = a + 5;
```

becomes:

```
int b = 10 + 5;
```

and then constant folding can produce `b = 15`.

Why it matters: It converts variable-based expressions into constant expressions and opens the door for folding, branch elimination, and dead-code elimination.

Where used: Optimizing compilers, JITs, interpreters with bytecode optimization, static analyzers, and query engines.

Why interviewers ask: It connects simple optimization with data-flow analysis.

2. Core Idea

Intuition: Track facts like "variable `a` currently equals `10`" and substitute that fact safely.

Analogy: If a restaurant menu says today's fixed lunch price is 100, every bill using "lunch price" can directly use 100 until the menu changes.

Example:

```
int a = 4;
int c = a * 2;
a = input();
int d = a * 2;
```

Steps:

1. `a = 4`, so `a` is constant.
2. Replace first `a * 2` with `4 * 2`.
3. After `a = input()`, `a` is no longer known.
4. Do not replace second `a`.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Local propagation	Within one basic block	Simple and fast	Straight-line code	Basic blocks
Global propagation	Across control flow	More powerful	Across <code>if</code> /loops	Data-flow analysis
Conditional propagation	Uses branch conditions	Removes branches	<code>if (x == 0)</code> then <code>x</code> known	Path sensitivity
Sparse propagation	Uses SSA form	Efficient in modern compilers	SSA constants	Compiler IR
Kill information	Assignment invalidates old constant	Prevents wrong substitution	<code>x = read()</code>	Correctness

4. Real-World Example

In backend server code, a feature flag compiled as a constant can make an optimizer remove unused branches:

```
const bool LOGGING = false;
if (LOGGING) write_log();
```

After propagation and folding, the branch can disappear.

5. Diagrams / Mental Models

```
x = 5      fact: x -> 5
y = x + 1  replace x with 5
x = read() kill fact: x is unknown
z = x + 1  cannot replace
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is constant propagation?	Replacing variables with known constants.	Known definitions, safe substitution	Saying it only evaluates math
How is it different from folding?	Propagation substitutes values; folding evaluates expressions.	Relationship between both	Merging both terms
Can it cross basic blocks?	Yes, with global data-flow analysis.	CFG, reaching definitions	Assuming only local
Why must assignments kill facts?	A new assignment changes the variable's value.	Correctness	Reusing stale constants
What happens at branches?	The compiler joins facts from possible paths.	Meet operation	Ignoring multiple paths
Can a variable be constant on one branch only?	Yes, conditional propagation may use path facts.	Path sensitivity	Treating all paths alike
Does propagation remove code directly?	Not always, but it enables folding and DCE.	Optimization pipeline	Claiming direct deletion only
What is SCCP?	Sparse conditional constant propagation works on SSA and reachable blocks.	SSA, reachability	Not knowing sparse idea
Can globals be propagated?	Sometimes, if immutability and visibility rules allow it.	Aliasing, external linkage	Assuming all globals constant
Why ask this in interviews?	It tests data-flow reasoning.	Definitions and uses	Giving only syntax example

7. Deep-Dive Questions

Question	Answer
Why does aliasing make propagation harder?	A pointer write may change a variable indirectly, so the compiler may lose certainty.
How does SSA help propagation?	Each variable version has one definition, making value tracking simpler.
What is the lattice for constant propagation?	Usually UNDEF , specific constants, and NAC meaning not a constant.
Why is propagation conservative?	If the compiler cannot prove a value, it must preserve original behavior.
How can propagation remove unreachable code?	A propagated condition can fold to true/false, making one branch unreachable.

8. Comparison Tables

Local Constant Propagation	Global Constant Propagation
Works inside one basic block	Works across CFG
No complex join needed	Needs data-flow equations
Faster, less powerful	Slower, more powerful
Good for straight-line code	Good for branches and loops

9. Common Mistakes

- Forgetting that reassignment kills constant facts.
- Replacing values across unknown function calls without checking side effects.
- Ignoring pointer aliases.
- Assuming constants are always source-level **const**.

10. Edge Cases / Special Cases

- **volatile** variables should not be propagated like ordinary constants.
- Global variables may change through other translation units or threads.
- Pointer writes can invalidate memory-based facts.
- Different paths may give different constants, producing "not a constant."

11. How to Explain in Interview

"Constant propagation tracks variables whose values are known constants and substitutes those constants at use sites. It often enables constant folding and dead-code elimination."

12. Quick Revision Notes

- Key definition: replace known-constant variables.
- Important point: assignments and aliases can kill facts.

- Common comparison: local vs global propagation.
- Interview trap: propagation must be proven on all relevant paths.

13. Practice Tasks

1. Trace constants through a basic block.
2. Propagate constants through an `if-else` and mark unknown joins.
3. Build a tiny map-based constant propagation pass for three-address code.
4. Explain why pointer writes make propagation difficult.

14. Final Cheat Sheet

Item	Answer
Core definition	Replace variable uses with known constants
Why it matters	Enables folding, branch removal, and simpler code
Most asked question	Difference from constant folding
Common comparison	Local vs global propagation
One-line answer	"It carries known constant values forward and substitutes them safely."

3. Dead-Code Elimination

1. Overview

Definition: Dead-code elimination, or DCE, removes code that does not affect the program's observable output.

Why it matters: It reduces code size, improves cache behavior, removes useless computation, and cleans up after other optimizations.

Where used: Native compilers, Java bytecode optimizers, JavaScript bundlers, link-time optimizers, database execution plans, and mobile build tools.

Why interviewers ask: It tests whether you understand side effects, liveness, reachability, and observable behavior.

2. Core Idea

Intuition: If a computation's result is never used, or a block can never run, remove it.

Analogy: If a chef chops vegetables that never go into any dish, that work can be skipped.

Example:

```
int x = 10;
x = 20;
printf("%d", x);
```

The first assignment is dead because `10` is overwritten before being read.

Optimized:

```
int x = 20;
printf("%d", x);
```

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Unreachable code	Code never executed	Reduces size	after <code>return</code>	CFG reachability
Dead stores	Writes never read	Removes wasted memory writes	<code>x=1; x=2;</code>	Live variables
Side effects	Observable actions	Must preserve behavior	<code>printf</code> , I/O	Correctness
Control-dependent DCE	Branch removal after condition folding	Cleans CFG	<code>if(false)</code>	Constant folding link
Aggressive DCE	Removes code not contributing to outputs	More global	SSA-based DCE	Roots of liveness

4. Real-World Example

JavaScript bundlers remove unused exports from frontend applications. If a utility function is imported nowhere, tree shaking can remove it from the production bundle, reducing load time.

5. Diagrams / Mental Models

```
Instruction produces value
  |
  v
Is value used later OR instruction has side effect?
  |
no  v
Remove instruction
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is dead code?	Code that cannot affect observable behavior.	Unused or unreachable	Saying "code after comments"
What is a dead store?	Assignment whose value is never read.	Liveness	Removing stores with side effects
Can function calls be removed?	Only if proven side-effect-free and result unused.	Purity	Removing arbitrary calls
What is unreachable code?	Code no execution path can reach.	CFG	Confusing with unused variable
How does DCE use liveness?	If a defined variable is not live after an instruction, the definition may be dead.	Live-out	Ignoring side effects
Does DCE change output?	Correct DCE must not.	Semantic preservation	Saying it changes logic
Why does folding help DCE?	Folded branches expose unreachable paths.	Optimization interaction	Viewing passes independently
Is <code>x++;</code> dead?	It may be dead if <code>x</code> unused and no volatile/overflow concerns.	Side effects and language rules	Always removing it
What is tree shaking?	DCE at module/bundle level.	Application build systems	Thinking only compilers do DCE
Why ask DCE?	It tests practical reasoning about useful computation.	Liveness and effects	Giving only one example

7. Deep-Dive Questions

Question	Answer
Why is memory DCE hard?	Aliasing means another pointer may read a store later.
What are roots in aggressive DCE?	Observable operations such as returns, stores to visible memory, calls with side effects, and I/O.
Can exception-throwing code be dead?	Not if removing it changes whether an exception is thrown.
How does SSA simplify DCE?	Use-def chains make it easy to find unused computed values.
Can a loop be removed?	Yes, if it has no side effects and its result is unused, but termination behavior must be considered.

8. Comparison Tables

Dead-Code Elimination	Unreachable-Code Elimination
Removes useless computations	Removes blocks that cannot execute
Uses liveness/use information	Uses CFG reachability
Example: unused assignment	Example: code after <code>return</code>
May require side-effect analysis	Usually follows branch simplification

9. Common Mistakes

- Removing code with I/O, volatile access, locks, or exceptions.
- Thinking all unused-looking code is dead.
- Ignoring memory aliasing.
- Forgetting that nontermination can be observable in some languages.

10. Edge Cases / Special Cases

- Infinite loops may not be removable if termination behavior is observable.
- Volatile memory operations are observable.
- Debug builds may keep code for diagnostics.
- Function calls require purity analysis before removal.

11. How to Explain in Interview

"Dead-code elimination removes instructions or blocks that do not affect observable behavior, such as overwritten assignments or unreachable branches, while preserving side effects."

12. Quick Revision Notes

- Key definition: remove useless code.
- Important point: observable behavior includes I/O, volatile, exceptions, and visible memory writes.
- Common comparison: dead store vs unreachable code.
- Trap: not every unused-looking call is removable.

13. Practice Tasks

1. Mark dead stores in a three-address-code block.
2. Build a liveness table and remove unused definitions.
3. Remove code after `return`.
4. Explain why `f()`; cannot be removed unless `f` is pure.

14. Final Cheat Sheet

Item	Answer
Core definition	Remove code with no observable effect
Why it matters	Smaller, faster generated code

Item	Answer
Most asked question	Can calls be removed?
Common comparison	Dead store vs unreachable code
One-line answer	"DCE deletes computations whose results are never needed and whose execution has no side effect."

4. Common Subexpression Elimination

1. Overview

Definition: Common subexpression elimination, or CSE, avoids recomputing the same expression when its operands have not changed.

Why it matters: It reduces repeated arithmetic, memory addressing, and expensive expression evaluation.

Where used: Compiler middle ends, database query optimizers, graphics compilers, and JIT engines.

Why interviewers ask: It tests expression equivalence, variable modification, and available expression reasoning.

2. Core Idea

Intuition: If you already computed $a + b$, and neither a nor b changed, reuse the old result.

Analogy: If you already calculated the total bill for the same cart, do not scan the cart again unless the cart changed.

Example:

```
x = a + b;  
y = (a + b) * 2;
```

Optimized:

```
t = a + b;  
x = t;  
y = t * 2;
```

Steps:

1. Identify repeated expression $a + b$.
2. Check no operand is modified between occurrences.
3. Store first result in temporary.

4. Replace later occurrence with temporary.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Local CSE	Within one basic block	Simple and common	repeated expression in straight-line code	Operand modification
Global CSE	Across basic blocks	More powerful	repeated after branches	Available expressions
Value numbering	Assign numbers to equivalent values	Detects equivalence	$a+b$ and $b+a$ for commutative ops	Compiler technique
Memory expressions	Loads may repeat	Hard due to aliasing	$*p + 1$	Side effects
Temporary cost	Reuse may need extra register	Can cause register pressure	storing t	Trade-off

4. Real-World Example

In a database query engine, a filter expression like $price * quantity$ may appear in both **WHERE** and **SELECT**. The optimizer can compute it once per row and reuse it.

5. Diagrams / Mental Models

```

expr = a + b
  |
  v
Have a and b changed since last same expr?
  |
  no v
Reuse previous result

```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is CSE?	Reusing a previously computed expression.	Same operands, no changes	Reusing unsafe expressions
When is $a+b$ common?	When the same value was computed and a, b unchanged.	Availability	Ignoring assignments
Difference between local and global CSE?	Local is within block; global crosses CFG.	Basic block vs CFG	Saying both same

Question	Clear Answer	Expected Points	Common Mistake
Can $a+b$ and $b+a$ match?	For commutative operations, if language rules allow.	Algebraic equivalence	Ignoring overflow/float
Can memory loads be CSE'd?	Sometimes, if no aliasing write changes memory.	Aliasing	Always reusing loads
What analysis supports CSE?	Available expressions and value numbering.	Data-flow	Saying liveness
Does CSE always improve speed?	Usually, but may increase register pressure.	Trade-off	Assuming always better
Is CSE same as memoization?	No, CSE is compile-time reuse within code; memoization is runtime caching.	Scope	Confusing runtime cache
What invalidates CSE?	Assignment to operands or side-effecting operation.	Kill set	Forgetting function calls
Why ask CSE?	It tests safe reuse of computed values.	Correctness	Only giving formula example

7. Deep-Dive Questions

Question	Answer
Why can floating-point CSE be restricted?	Reassociation or equivalence may change rounding or exception behavior.
How does value numbering help CSE?	It detects expressions that compute the same value even if syntactically different.
Why can CSE hurt performance?	Extra temporaries can increase register pressure and spills.
What is partial redundancy elimination?	It generalizes CSE by moving computations so partially repeated expressions become fully reusable.
How do function calls affect CSE?	Unknown calls may modify memory or globals, invalidating expression facts.

8. Comparison Tables

CSE	Constant Folding
Avoids repeated nonconstant expression computation	Evaluates constant expressions
Needs no operand changes between uses	Needs operands known at compile time
Example: reuse $a+b$	Example: $2+3 \rightarrow 5$
Can introduce temporaries	Usually replaces with literals

9. Common Mistakes

- Reusing expressions after operands changed.
- Ignoring function calls that may change memory.
- Treating floating-point algebra like integer algebra.
- Forgetting register pressure.

10. Edge Cases / Special Cases

- $a + b$ may not equal $b + a$ for floating-point in all optimization modes.
- Loads from volatile memory cannot be reused casually.
- Pointer aliasing can invalidate memory expressions.
- Reuse may be blocked if temporary lifetime is too costly.

11. How to Explain in Interview

"CSE detects repeated expressions and computes them once if the inputs have not changed, replacing later occurrences with the saved result."

12. Quick Revision Notes

- Key definition: eliminate repeated expression computation.
- Important point: expression must be available.
- Common comparison: local CSE vs global CSE.
- Trap: memory aliasing and floating-point semantics.

13. Practice Tasks

1. Find repeated expressions in a basic block.
2. Build available-expression sets for a small CFG.
3. Explain why $x = *p; y = *p;$ may not be CSE-safe after a function call.
4. Implement simple local CSE using a map from expression to temporary.

14. Final Cheat Sheet

Item	Answer
Core definition	Reuse repeated expressions when operands are unchanged
Why it matters	Avoids duplicate computation
Most asked question	What invalidates a common expression?
Common comparison	Local vs global CSE
One-line answer	"CSE computes the same expression once and reuses it safely."

5. Copy Propagation

1. Overview

Definition: Copy propagation replaces uses of a variable with another variable when the compiler knows they hold the same value due to a copy assignment.

```
x = y;  
z = x + 1;
```

becomes:

```
z = y + 1;
```

Why it matters: It removes unnecessary temporaries and often exposes dead assignments.

Where used: Three-address-code optimization, SSA simplification, register allocation preparation, JIT compilers.

Why interviewers ask: It tests use-def chains, assignment validity, and interaction with DCE.

2. Core Idea

Intuition: If x is just another name for y , use y directly until either value may change.

Analogy: If a teammate says "call Rahul using my phone number" and then gives you Rahul's actual number, you can use the actual number directly.

Example:

```
t1 = a;  
b = t1 + 5;
```

Optimized:

```
b = a + 5;
```

Then $t1 = a$ may become dead and removable.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Copy statement	Assignment $x = y$	Source of propagation	$t = a$	Definition-use

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Kill condition	Reassignment invalidates copy	Prevents wrong replacement	<code>y = 9</code>	Correctness
Local copy propagation	Within a block	Simple	straight-line TAC	Basic blocks
Global copy propagation	Across blocks	More useful	copies across branches	Data-flow
DCE interaction	Copy becomes unused	Cleans temporaries	remove <code>t=a</code>	Pass ordering

4. Real-World Example

Intermediate code often creates temporaries:

```
t1 = user_id
t2 = t1
call load_user(t2)
```

Copy propagation can simplify this to use `user_id` directly, reducing register moves.

5. Diagrams / Mental Models

```
x = y      copy fact: x == y
use x      replace with y
x = ...    kill x == y
y = ...    kill x == y
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is copy propagation?	Replacing a copied variable with its source.	<code>x=y</code> , substitute uses	Confusing with constant propagation
How does it help?	Removes moves and exposes dead code.	DCE interaction	Saying it only saves memory
When is it unsafe?	After source or destination may change.	Kill conditions	Ignoring reassignment
Is it local or global?	Can be both.	Scope	Assuming only local
What code often creates copies?	Three-address code, SSA lowering, register moves.	Compiler IR	Thinking programmers write all copies

Question	Clear Answer	Expected Points	Common Mistake
Can copy propagation change aliases?	It must respect memory and pointer semantics.	Aliasing	Treating pointer copies as value copies
What analysis supports it?	Reaching copies or use-def information.	Data-flow	Saying available expressions
Does it remove the original assignment?	Not directly; DCE removes it if unused.	Pass interaction	Mixing passes
What is the difference from CSE?	CSE reuses expressions; copy propagation replaces equivalent variables.	Expression vs variable	Calling them same
Why ask it?	It checks understanding of temporaries and data flow.	IR-level thinking	Only source-level answer

7. Deep-Dive Questions

Question	Answer
Why can copy propagation improve register allocation?	Fewer artificial moves may reduce register pressure and move instructions.
How does SSA affect copy propagation?	Phi/copy coalescing and SSA renaming make many copies easier to reason about.
What is copy coalescing?	Register allocator assigns copy-related variables to the same register to remove moves.
Why can propagation increase live ranges?	Replacing uses may keep the source variable live longer.
What happens at control-flow joins?	Copy facts must hold on all incoming paths to be safely propagated generally.

8. Comparison Tables

Copy Propagation	Constant Propagation
Replaces variable with another variable	Replaces variable with constant
Example: $x=y$; $z=x \rightarrow z=y$	Example: $x=5$; $z=x \rightarrow z=5$
Killed by changes to either variable	Killed by changes to variable
Often removes moves	Often enables folding

9. Common Mistakes

- Propagating after source variable changes.

- Assuming copy propagation itself deletes assignments.
- Ignoring increased live ranges.
- Confusing variable equality with pointer target equality.

10. Edge Cases / Special Cases

- Volatile variables restrict substitution.
- Pointers and references require alias awareness.
- At CFG joins, the same copy must reach from all relevant paths.
- Debug info may preserve variable names even if code is optimized.

11. How to Explain in Interview

"Copy propagation removes unnecessary temporary variables by replacing a copied variable with its original source wherever the copy is still valid."

12. Quick Revision Notes

- Key definition: replace `x` with `y` after `x = y`.
- Important point: killed when either variable changes.
- Common comparison: copy vs constant propagation.
- Trap: original copy deletion is DCE's job.

13. Practice Tasks

1. Simplify three-address code with copy propagation.
2. Mark where copy facts are killed.
3. Show how DCE removes the now-unused copy assignment.
4. Trace copies through an `if-else`.

14. Final Cheat Sheet

Item	Answer
Core definition	Replace copied variables with their source
Why it matters	Removes temporary moves and simplifies IR
Most asked question	When is it unsafe?
Common comparison	Copy propagation vs constant propagation
One-line answer	"If <code>x</code> is only a copy of <code>y</code> , use <code>y</code> directly until the fact is killed."

6. Strength Reduction

1. Overview

Definition: Strength reduction replaces expensive operations with cheaper equivalent operations.

Example:

```
x = i * 8;
```

may become:

```
x = i << 3;
```

or inside loops, repeated multiplication may become repeated addition.

- Why it matters:** It reduces CPU cost, especially in loops.
- Where used:** Loop optimizers, embedded compilers, numeric kernels, graphics compilers, database engines.
- Why interviewers ask:** It tests cost models, arithmetic equivalence, and loop optimization intuition.

2. Core Idea

- Intuition:** Use a cheaper operation that gives the same result.
- Analogy:** If you need to count money in groups of 10 repeatedly, keep adding 10 instead of recounting from zero each time.
- Example:**

```
for (int i = 0; i < n; i++) {  
    a[i] = i * 4;  
}
```

Optimized idea:

```
int t = 0;  
for (int i = 0; i < n; i++) {  
    a[i] = t;  
    t += 4;  
}
```

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Arithmetic strength reduction	Replace costly arithmetic	Faster instructions	multiply to shift	Valid only for powers of two
Induction variables	Variables changing regularly in loops	Main loop use case	i, i*4	Loop analysis

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Address calculation	Turn index math into pointer increments	Common in arrays	<code>base + i*4</code>	Machine code
Cost model	Cheaper depends on target CPU	Avoid bad optimization	modern multiply may be fast	Architecture awareness
Correctness rules	Preserve overflow and sign behavior	Avoid wrong code	signed shifts	Language semantics

4. Real-World Example

In array traversal, compilers often replace repeated address multiplication with pointer increments:

```
for (i = 0; i < n; i++) sum += a[i];
```

Machine code may maintain a pointer that advances by element size instead of recomputing `base + i * size`.

5. Diagrams / Mental Models

```

Expensive repeated operation
  |
  v
Can result be updated incrementally?
  |
yes  v
Use cheaper recurrence

```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is strength reduction?	Replacing expensive operations with cheaper equivalents.	Cost, equivalence	Only saying multiply to shift
Why useful in loops?	Savings repeat many times.	Loop hot paths	Ignoring loop frequency
Give an example.	<code>i*4</code> can become incremental addition or shift.	Simple example	Unsafe example
Is multiplication always expensive?	Not on all CPUs; compilers use cost models.	Target dependence	Old assumption always true

Question	Clear Answer	Expected Points	Common Mistake
What is an induction variable?	Variable updated by a fixed pattern each iteration.	Loop analysis	Only loop counter
Can division be strength-reduced?	Sometimes, e.g., division by constant using multiply/shift.	Magic constants	Saying always shift
What can go wrong with shifts?	Signedness, overflow, rounding for negatives.	Correctness	Treating <code>/2</code> as <code>>>1</code> always
Is it machine independent?	The idea is general, profitability is target-specific.	Cost model	Assuming universal benefit
How relates to LICM?	LICM moves invariant work; strength reduction replaces costly recurrence work.	Loop optimization distinction	Mixing both
Why ask it?	It tests practical performance reasoning.	Loops and CPU cost	Giving theoretical answer only

7. Deep-Dive Questions

Question	Answer
Why is <code>x / 2</code> not always <code>x >> 1</code> ?	For signed negative values, rounding rules may differ.
How do compilers optimize division by constants?	They may use multiplication by a precomputed reciprocal plus shifts.
What is an induction variable family?	Related values derived from the same loop counter, such as <code>i</code> , <code>4*i</code> , and <code>base+4*i</code> .
Can strength reduction hurt register pressure?	Yes, extra recurrence variables may need registers.
How does target architecture affect it?	Some instructions are already cheap, so replacement may not be profitable.

8. Comparison Tables

Strength Reduction	Loop Invariant Code Motion
Replaces expensive operation with cheaper one	Moves repeated invariant operation out of loop
Example: multiply to add	Example: move <code>n*4</code> before loop
Focuses on operation cost	Focuses on repetition
Often uses induction variables	Uses invariance analysis

9. Common Mistakes

- Assuming shift always equals multiplication/division.
- Ignoring signed integer behavior.
- Forgetting modern CPU cost models.
- Adding extra variables that increase register pressure.

10. Edge Cases / Special Cases

- Floating-point transformations may change precision.
- Overflow behavior must match source language.
- Division by constants needs careful rounding handling.
- Embedded systems may benefit more than desktop CPUs.

11. How to Explain in Interview

"Strength reduction replaces costly operations with cheaper equivalent ones, especially inside loops, such as replacing repeated multiplication by an induction variable with incremental addition."

12. Quick Revision Notes

- Key definition: expensive operation to cheaper operation.
- Important point: equivalence depends on types and target.
- Common comparison: strength reduction vs LICM.
- Trap: signed division and shifts.

13. Practice Tasks

1. Convert loop multiplication into incremental addition.
2. Identify induction variables in a nested loop.
3. Check whether `x / 2 -> x >> 1` is safe for signed integers.
4. Inspect generated assembly for `i * 8`.

14. Final Cheat Sheet

Item	Answer
Core definition	Replace expensive operations with cheaper equivalents
Why it matters	Big savings in hot loops
Most asked question	Is multiply-to-shift always safe?
Common comparison	Strength reduction vs LICM
One-line answer	"Use a cheaper equivalent operation, but only when language semantics stay the same."

7. Loop Invariant Code Motion

1. Overview

Definition: Loop invariant code motion, or LICM, moves computations outside a loop when their result does not change across iterations.

Why it matters: Loops execute repeatedly, so moving one repeated calculation outside can save significant runtime.

Where used: Native compilers, JIT compilers, numeric computing, graphics, databases, and backend services.

Why interviewers ask: It tests loop analysis, dominance, side effects, and safety.

2. Core Idea

Intuition: If a value is the same every iteration, compute it once before the loop.

Analogy: If every student in a class needs the same exam instructions, print them once on the board instead of repeating them to each student.

Example:

```
for (int i = 0; i < n; i++) {  
    a[i] = b[i] + x * y;  
}
```

If x and y do not change:

```
int t = x * y;  
for (int i = 0; i < n; i++) {  
    a[i] = b[i] + t;  
}
```

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Loop invariant expression	Same value every iteration	Candidate for motion	$x*y$	Operand definitions
Preheader	Block before loop	Safe placement	insert $t=x*y$	CFG transformation
Dominance	Definition reaches uses	Ensures availability	computed before all uses	Compiler theory
Side effects	Motion can change behavior	Must preserve order	function call in loop	Safety
Speculation	Moving may execute code that original loop skipped	Can be unsafe	zero iterations	Exceptions/traps

4. Real-World Example

In a database scan, a query may compare each row against a constant expression:

```
WHERE order_total > tax_rate * threshold
```

The database engine can compute `tax_rate * threshold` once before scanning rows.

5. Diagrams / Mental Models

```
Before:
preheader -> loop [compute same value every time]

After:
preheader [compute once] -> loop [reuse value]
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is LICM?	Moving loop-invariant computation outside the loop.	Same value each iteration	Moving changing values
What is loop invariant?	Expression whose operands do not change in the loop.	Definitions inside/outside loop	Only constants
Why use a preheader?	It gives a single safe place before loop entry.	CFG	Ignoring placement
Can function calls be moved?	Only if pure and safe to execute earlier.	Side effects	Moving all calls
What if loop runs zero times?	Moving trapping/side-effect code may change behavior.	Speculation safety	Ignoring zero iterations
What analysis is needed?	Loop detection, dominance, side-effect/alias analysis.	Compiler fundamentals	Saying only syntax scan
Does LICM reduce complexity?	It reduces repeated work; asymptotic may stay same.	Performance nuance	Claiming $O(n)$ to $O(1)$ always
How differs from strength reduction?	LICM moves invariant work; strength reduction changes operation form.	Distinction	Mixing loop optimizations
Can memory loads be invariant?	Yes if memory cannot be modified in loop.	Alias analysis	Assuming loads always invariant

Question	Clear Answer	Expected Points	Common Mistake
Why ask it?	It tests practical loop optimization safety.	Invariance and effects	Example-only answer

7. Deep-Dive Questions

Question	Answer
Why is dominance important?	The moved definition must dominate all uses that depend on it.
What is loop preheader insertion?	Creating a block that always runs immediately before loop entry.
Can invariant code be moved out of nested loops?	Yes, to the outermost loop where it remains invariant and safe.
Why does alias analysis matter?	A store inside the loop may change a loaded value.
How do exceptions restrict LICM?	Moving an expression that can throw may throw even when original loop body would not execute.

8. Comparison Tables

LICM	Loop Unrolling
Moves repeated invariant work out	Duplicates loop body to reduce overhead
Usually reduces instructions executed	May increase code size
Needs invariance proof	Needs trip-count/profitability analysis
Example: $x*y$ before loop	Example: process 4 items per iteration

9. Common Mistakes

- Moving code that depends on loop variable.
- Moving side-effecting function calls.
- Ignoring zero-iteration loops.
- Assuming every load from outside loop is invariant.

10. Edge Cases / Special Cases

- Trapping operations may not be safe to speculate.
- Volatile loads cannot be moved freely.
- Multi-threaded shared memory can restrict motion.
- Alias analysis decides whether memory values are stable.

11. How to Explain in Interview

"LICM finds computations inside a loop whose value stays the same across iterations and moves them before the loop, usually into a preheader, if doing so is safe."

12. Quick Revision Notes

- Key definition: move invariant computation out of loop.
- Important point: safety includes side effects and exceptions.
- Common comparison: LICM vs strength reduction.
- Trap: loop may execute zero times.

13. Practice Tasks

1. Mark invariant expressions in a loop.
2. Add a loop preheader in a small CFG.
3. Explain why `read()` cannot be moved out.
4. Optimize a nested loop by hoisting outer-invariant code.

14. Final Cheat Sheet

Item	Answer
Core definition	Hoist loop-invariant computations
Why it matters	Saves repeated work in loops
Most asked question	When is hoisting unsafe?
Common comparison	LICM vs loop unrolling
One-line answer	"Compute loop-stable values once before the loop instead of every iteration."

8. Loop Unrolling

1. Overview

Definition: Loop unrolling duplicates the loop body to reduce loop-control overhead and expose instruction-level optimization.

Why it matters: It can reduce branch overhead, improve scheduling, and enable vectorization.

Where used: Numeric libraries, image processing, machine learning kernels, embedded code, JavaScript JITs, and C/C++ compilers.

Why interviewers ask: It tests performance trade-offs: speed vs code size.

2. Core Idea

Intuition: Do more work per loop iteration so the loop checks and jumps happen fewer times.

Analogy: Instead of carrying one book at a time from one shelf to another, carry four books per trip.

Example:

```
for (int i = 0; i < n; i++) sum += a[i];
```

Unrolled by 4:

```
int i = 0;
for (; i + 3 < n; i += 4) {
    sum += a[i] + a[i+1] + a[i+2] + a[i+3];
}
for (; i < n; i++) sum += a[i];
```

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Unroll factor	Number of body copies	Controls speed/code-size tradeoff	factor 4	Profitability
Remainder loop	Handles leftover iterations	Correctness for any n	$n \% 4$	Edge cases
Full unrolling	Remove loop entirely	Good for small fixed counts	for $i < 4$	Code size
Partial unrolling	Reduce but keep loop	Common for large loops	$i += 4$	Practical compilers
Vectorization support	Exposes adjacent operations	Helps SIMD	4 loads at once	Auto-vectorization

4. Real-World Example

Image processing loops often operate on pixels. Unrolling lets the CPU process several pixels per iteration, reducing branch overhead and helping SIMD instructions process multiple color values.

5. Diagrams / Mental Models

```
Normal loop:
check -> work1 -> jump
check -> work2 -> jump
check -> work3 -> jump
check -> work4 -> jump

Unrolled:
check -> work1 work2 work3 work4 -> jump
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is loop unrolling?	Duplicating loop body to reduce loop overhead.	Fewer branches	Saying it changes algorithm
Why does it help?	Less branch overhead and more scheduling/vectorization opportunity.	CPU pipeline	Only branch count
What is unroll factor?	Number of iterations combined.	Factor 2/4/8	Ignoring code size
What is a remainder loop?	Handles iterations not divisible by factor.	Correctness	Forgetting leftover items
Does it always improve performance?	No, code size and cache pressure can hurt.	Trade-off	Saying always faster
What is full unrolling?	Expanding all iterations when count is known and small.	Fixed trip count	Applying to huge loops
How relates to vectorization?	It exposes independent adjacent operations.	SIMD	Treating as same thing
Can unrolling affect instruction cache?	Yes, larger code can hurt locality.	Code-size tradeoff	Ignoring cache
Who chooses unroll factor?	Compiler using heuristics/profile or programmer pragmas.	Cost model	Arbitrary choice
Why ask it?	It tests practical optimization trade-offs.	Speed vs size	Only textbook definition

7. Deep-Dive Questions

Question	Answer
Why can unrolling improve instruction scheduling?	More independent operations are visible to the scheduler.
How can unrolling hurt branch prediction?	Fewer branches usually help, but bigger code may reduce locality.
What is loop peeling?	Separating a few iterations to handle alignment or special cases.
What is loop unswitching?	Moving a loop-invariant condition outside by creating separate loops.
Why do compilers need trip-count analysis?	To know whether unrolling is correct and profitable.

8. Comparison Tables

Loop Unrolling	Auto-Vectorization
Duplicates scalar loop body	Uses SIMD instructions
Reduces branch overhead	Processes multiple data lanes at once
May help vectorization	May require alignment/dependence proof
Increases code size	Uses hardware vector registers

9. Common Mistakes

- Forgetting the remainder loop.
- Thinking unrolling reduces Big-O complexity.
- Ignoring instruction-cache pressure.
- Using huge unroll factors blindly.

10. Edge Cases / Special Cases

- Unknown trip counts require cleanup loops.
- Very small loops may be fully unrolled.
- Loops with dependencies may not benefit.
- Embedded systems may reject unrolling due to code size.

11. How to Explain in Interview

"Loop unrolling reduces loop overhead by executing multiple original iterations inside one new iteration, but it trades speed for larger code size."

12. Quick Revision Notes

- Key definition: duplicate loop body.
- Important point: handle leftovers.
- Common comparison: unrolling vs vectorization.
- Trap: bigger code can be slower.

13. Practice Tasks

1. Unroll a loop by factor 2 and factor 4.
2. Add remainder handling for arbitrary n .
3. Compare assembly for normal and unrolled loops.
4. Explain when unrolling helps vectorization.

14. Final Cheat Sheet

Item	Answer
Core definition	Combine multiple iterations into one
Why it matters	Reduces loop overhead and exposes parallelism
Most asked question	What about leftover iterations?

Item	Answer
Common comparison	Loop unrolling vs vectorization
One-line answer	"Unrolling does more work per iteration to reduce loop-control cost."

9. Peephole Optimization

1. Overview

Definition: Peephole optimization examines a small window of generated instructions and replaces inefficient instruction patterns with better ones.

Why it matters: It cleans up low-level code after instruction selection and register allocation.

Where used: Machine-code generators, assemblers, bytecode optimizers, embedded compilers, VM JITs.

Why interviewers ask: It bridges compiler theory and assembly-level practical optimization.

2. Core Idea

Intuition: Look at a few neighboring instructions and simplify obvious waste.

Analogy: After writing an essay, you scan nearby words and replace "in order to" with "to."

Example:

```
MOV R1, R2
MOV R2, R1
```

may be redundant depending on surrounding context. Another simple example:

```
ADD R1, 0
```

can be removed if flags are not needed.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Redundant instruction removal	Delete no-op work	Smaller code	ADD R, 0	Flags matter
Algebraic simplification	Use simpler instruction	Faster code	MUL R, 2 -> SHL R, 1	Semantics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Jump optimization	Remove unnecessary jumps	Better control flow	jump to next instruction	CFG cleanup
Machine idioms	Use target-specific efficient pattern	Faster on CPU	zero register with XOR	Architecture
Window size	Number of instructions inspected	Limits power	2-5 instructions	Local nature

4. Real-World Example

A JVM or Python bytecode optimizer may replace:

```
LOAD_CONST 1
LOAD_CONST 2
BINARY_ADD
```

with:

```
LOAD_CONST 3
```

or remove jumps that target the next instruction.

5. Diagrams / Mental Models

```
Instruction stream:
[i1][i2][i3][i4][i5]
      ^ small window ^

If pattern matches -> replace with shorter/faster pattern
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is peephole optimization?	Small-window low-level pattern replacement.	Local instruction patterns	Describing global analysis
Why called peephole?	Compiler looks through a small window of instructions.	Window idea	Literal source-code-only answer
Give examples.	Remove no-ops, combine moves, simplify jumps.	Assembly-level examples	Only high-level examples

Question	Clear Answer	Expected Points	Common Mistake
When is <code>ADD R, 0</code> removable?	If condition flags or side effects are not needed.	Machine semantics	Always removing it
What is jump optimization?	Removing or redirecting unnecessary branches.	Control-flow cleanup	Ignoring labels
Is it machine dependent?	Often yes, because instruction costs differ.	Target-specific	Treating all CPUs same
Does it need data-flow analysis?	Usually limited, but may use local liveness/flags info.	Local nature	Saying no analysis ever
Can peephole run on bytecode?	Yes, any instruction-like representation.	VM bytecode	Only native assembly
How differs from CSE?	CSE is expression-level; peephole is local instruction pattern-level.	IR level	Mixing layers
Why ask it?	It tests practical code generation knowledge.	Backend compiler	Only saying "makes code faster"

7. Deep-Dive Questions

Question	Answer
Why are CPU flags important?	An instruction that seems useless may set flags used by a later branch.
What is superoptimization?	Searching for the best equivalent instruction sequence, a generalized peephole idea.
Why can target idioms matter?	Some CPUs execute special instruction patterns faster or encode them smaller.
How can peephole optimization affect debugging?	It may remove or reorder low-level instructions, complicating source mapping.
Can peephole optimization be unsafe?	Yes, if pattern replacement ignores exact machine semantics.

8. Comparison Tables

Peephole Optimization	Global Optimization
Small local instruction window	Whole function/program CFG
Usually backend-level	Usually IR-level
Pattern-based	Analysis-based
Fast and simple	More powerful but expensive

9. Common Mistakes

- Forgetting condition flags.
- Assuming local pattern replacement understands whole-program behavior.
- Applying source-level examples only.
- Ignoring target architecture.

10. Edge Cases / Special Cases

- **ADD 0** may set flags, so removal can be wrong.
- Redundant-looking loads/stores may involve volatile memory.
- Branch relaxation depends on instruction encoding distance.
- Pipeline behavior may make smaller code not always faster.

11. How to Explain in Interview

"Peephole optimization scans a small sequence of low-level instructions and replaces inefficient patterns with equivalent shorter or faster ones."

12. Quick Revision Notes

- Key definition: local low-level pattern optimization.
- Important point: machine semantics matter.
- Common comparison: peephole vs global optimization.
- Trap: flags and volatile operations.

13. Practice Tasks

1. Remove redundant assembly instructions from a short sequence.
2. Identify jump-to-next-instruction cases.
3. Explain why **CMP R,0** cannot always replace other tests.
4. Write three peephole rewrite rules.

14. Final Cheat Sheet

Item	Answer
Core definition	Optimize small instruction windows
Why it matters	Cleans low-level generated code
Most asked question	Give examples and safety issues
Common comparison	Peephole vs global optimization
One-line answer	"It is local pattern cleanup on generated instructions."

10. Local vs Global Optimization

1. Overview

Definition: Local optimization works within one basic block; global optimization works across multiple basic blocks in a control-flow graph.

Why it matters: Local optimization is simpler, but global optimization catches more real performance opportunities.

Where used: Every optimizing compiler pipeline.

Why interviewers ask: This is a core compiler-design distinction behind many optimization passes.

2. Core Idea

Intuition: Local optimization sees one straight-line region; global optimization sees branches, loops, and joins.

Analogy: Local optimization is organizing one room. Global optimization is organizing the whole house, including hallways and shared storage.

Example:

```
if (flag) x = 5;
else x = 5;
y = x + 1;
```

A local optimizer may not connect both branches. A global optimizer can see that `x` is `5` after the `if` and optimize `y = 6`.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Basic block	Straight-line code	Unit of local optimization	no internal jumps	CFG construction
CFG	Graph of basic blocks	Required for global optimization	branch edges	Control flow
Local analysis	No path joins	Fast	local CSE	Simplicity
Global analysis	Uses data-flow across blocks	Powerful	reaching definitions	Fixed point
Interprocedural scope	Across functions	Beyond global within function	inlining	IPO distinction

4. Real-World Example

A database query optimizer may locally simplify one predicate, but global planning considers joins, indexes, and filters across the full query plan.

5. Diagrams / Mental Models

```

Local:
[ basic block only ]

Global:
  [B1]
  /  \
[B2] [B3]
  \  /
  [B4]

```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is local optimization?	Optimization inside one basic block.	Straight-line code	Saying inside one function
What is global optimization?	Optimization across CFG blocks within a procedure.	CFG	Saying whole program only
What is a basic block?	Maximal straight-line code with single entry and exit.	Entry/exit	Any group of lines
Why is global harder?	Multiple paths require conservative joins.	Data-flow	Only "more code"
Give local optimization.	Local CSE, local constant folding.	Basic examples	Using interprocedural example
Give global optimization.	Global constant propagation, LICM.	CFG examples	Same as local
Is global same as interprocedural?	No, global often means within one procedure across blocks.	Terminology	Confusing scopes
Why use local at all?	It is cheap and catches many cases.	Compile-time cost	Thinking global replaces local
What data structure supports global?	Control-flow graph.	CFG	AST only
Why ask this?	It tests compiler IR and analysis scope.	Scope clarity	Vague "better optimization"

7. Deep-Dive Questions

Question	Answer
Why do joins lose precision?	Facts must be valid for multiple incoming paths.

Question	Answer
What is a fixed point?	A stable solution where further data-flow iterations do not change facts.
Can local optimizations run before global ones?	Yes, compilers often run many passes repeatedly.
What is loop-level optimization?	A global optimization focused on loop structures in the CFG.
How does SSA help global optimization?	It makes definitions explicit and joins represented by phi nodes.

8. Comparison Tables

Local Optimization	Global Optimization
One basic block	Multiple blocks in a CFG
Simple and fast	More complex
No branch joins	Handles branches and loops
Less powerful	More powerful
Example: local CSE	Example: LICM, global propagation

9. Common Mistakes

- Saying global always means whole-program.
- Calling any function-level optimization local.
- Forgetting basic block definition.
- Ignoring compile-time cost.

10. Edge Cases / Special Cases

- Some optimizations have local and global versions.
- Interprocedural optimization goes beyond normal global optimization.
- Exception edges complicate CFGs.
- Indirect jumps and dynamic dispatch can reduce precision.

11. How to Explain in Interview

"Local optimization works within a basic block, while global optimization uses the control-flow graph to optimize across branches and loops inside a procedure."

12. Quick Revision Notes

- Key definition: local is basic-block scope; global is CFG scope.
- Important point: global needs data-flow.
- Common comparison: local vs global vs interprocedural.
- Trap: global does not always mean whole program.

13. Practice Tasks

1. Divide code into basic blocks.
2. Draw a CFG for an `if-else` and loop.
3. Identify which optimizations require global analysis.
4. Solve a small reaching-definitions problem.

14. Final Cheat Sheet

Item	Answer
Core definition	Local: one block; global: across CFG
Why it matters	Determines optimization power and cost
Most asked question	Difference between local and global
Common comparison	Local vs global vs interprocedural
One-line answer	"Local sees straight-line code; global sees the control-flow graph."

11. Data-Flow Analysis

1. Overview

Definition: Data-flow analysis is a compiler technique for collecting facts about how values move through a program's control-flow graph.

Why it matters: Many optimizations need facts such as "which definitions reach here?", "is this variable live?", or "is this expression already available?"

Where used: Optimizers, static analyzers, IDE warnings, security scanners, and bug-finding tools.

Why interviewers ask: It is the mathematical foundation behind many compiler optimizations.

2. Core Idea

Intuition: Each basic block receives facts, changes them, and passes facts to successors until information stabilizes.

Analogy: Rumors spreading in a campus: each classroom receives information, adds/removes some, and passes it along paths until everyone has consistent knowledge.

Example:

```
B1: x = 1
B2: y = x + 2
```

Fact after **B1**: definition `x=1` reaches **B2**.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
CFG	Graph of basic blocks	Data flows along edges	<code>if</code> , loop edges	Build CFG
IN/OUT sets	Facts entering/leaving block	Core equations	<code>IN[B]</code> , <code>OUT[B]</code>	Transfer functions
Gen/Kill	Facts created/removed	Models block effect	assignment kills old defs	Equations
Forward analysis	Facts flow with execution	Reaching definitions	from entry to exit	Direction
Backward analysis	Facts flow backward	Live variables	from use to definition	Direction
May/Must analysis	Possible vs guaranteed facts	Precision and safety	reaching vs available	Meet operator

4. Real-World Example

An IDE warning "variable assigned but never used" uses live-variable-style data-flow analysis. It checks whether a value written to a variable is ever read later.

5. Diagrams / Mental Models

```

    facts in
      |
      v
    [basic block]
    gen facts
    kill facts
      |
      v
    facts out
  
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is data-flow analysis?	Computing program facts over CFG paths.	CFG, facts, fixed point	Saying it is data structures
What are IN and OUT sets?	Facts before and after a block.	Block equations	Confusing with input/output
What are gen and kill sets?	Facts generated and invalidated by a block.	Transfer function	Only definitions

Question	Clear Answer	Expected Points	Common Mistake
What is forward analysis?	Facts flow in execution direction.	Reaching definitions	Using liveness example
What is backward analysis?	Facts flow opposite execution.	Live variables	Reversing definitions
What is may analysis?	Fact may be true on some path.	Union meet	Saying guaranteed
What is must analysis?	Fact true on all paths.	Intersection meet	Saying possible
Why fixed point?	Loops need repeated solving until stable.	Iteration	Single pass always
Which optimizations use it?	DCE, CSE, constant propagation, LICM.	Applications	Only one pass
Why ask it?	It underlies compiler optimization.	Foundation	Memorized equations only

7. Deep-Dive Questions

Question	Answer
Why do loops require iteration?	Facts can circulate through back edges, so a single pass may miss stable information.
What is a lattice?	An ordered set of facts with meet operations used to combine information safely.
Why are monotone transfer functions important?	They help guarantee fixed-point convergence on finite lattices.
What is conservative analysis?	When uncertain, choose facts that preserve correctness even if less optimized.
How does SSA change data-flow?	It makes many def-use relationships explicit, simplifying several analyses.

8. Comparison Tables

Forward Analysis	Backward Analysis
Entry to exit direction	Exit to entry direction
Facts depend on earlier code	Facts depend on later uses
Example: reaching definitions	Example: live-variable analysis
Uses predecessors to compute IN	Uses successors to compute OUT
May Analysis	Must Analysis
True on at least one path	True on all paths

May Analysis	Must Analysis
Usually combines with union	Usually combines with intersection
Example: reaching definitions	Example: available expressions
More permissive facts	More guaranteed facts

9. Common Mistakes

- Memorizing equations without knowing direction.
- Confusing may and must analysis.
- Ignoring loops and fixed points.
- Forgetting that analyses are conservative.

10. Edge Cases / Special Cases

- Exception edges add hidden CFG paths.
- Function calls can kill many memory facts.
- Unreachable blocks can distort analysis if not handled.
- Pointers and aliasing reduce precision.

11. How to Explain in Interview

"Data-flow analysis computes facts over a control-flow graph using IN/OUT sets and transfer equations, usually iterating to a fixed point so optimizations can be applied safely."

12. Quick Revision Notes

- Key definition: facts over CFG paths.
- Important point: direction and meet operator matter.
- Common comparison: forward vs backward, may vs must.
- Trap: loops need fixed-point iteration.

13. Practice Tasks

1. Draw a CFG and compute IN/OUT sets manually.
2. Solve reaching definitions for a four-block program.
3. Solve live variables backward.
4. Identify gen and kill sets for assignments.

14. Final Cheat Sheet

Item	Answer
Core definition	Framework for computing program facts over CFG
Why it matters	Enables safe optimization
Most asked question	Forward vs backward analysis
Common comparison	May vs must analysis

Item	Answer
One-line answer	"Data-flow analysis tracks facts through control flow until they stabilize."

12. Reaching Definitions

1. Overview

Definition: A definition of variable `x` reaches a program point if there is a path from that definition to the point with no later redefinition of `x`.

Why it matters: It tells the compiler which assignments may provide a variable's current value.

Where used: Constant propagation, copy propagation, use-def chains, warnings, and program slicing.

Why interviewers ask: It is a classic forward may data-flow analysis.

2. Core Idea

Intuition: Ask: "Which assignments could be responsible for this variable value here?"

Analogy: If several people could have last edited a document, reaching definitions tells you whose edit may be the current one.

Example:

```
x = 1;    // d1
if (c) x = 2; // d2
y = x;    // d1 or d2 may reach here
```

Both `d1` and `d2` reach `y = x` because either path is possible.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Definition	Assignment to variable	Basic fact tracked	<code>x=1</code>	Identify defs
Reach	Path without redefinition	Determines possible values	<code>d1</code> reaches use	CFG paths
Kill	Redefinition removes old defs	Correctness	<code>x=2</code> kills old <code>x</code> defs	Kill set
May analysis	Definition may reach	Uses union	branch paths	Meet operator
Forward direction	Facts move with execution	Natural for definitions	entry to exit	Equations

4. Real-World Example

A compiler warning "variable may be uninitialized" checks whether a valid definition reaches a use on every path. Reaching definitions helps identify possible missing assignments.

5. Diagrams / Mental Models

```

d1: x = 1
  |
  v
d2: x = 2   kills d1 for x on this path
  |
  v
use x       reaching def: d2

```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is a reaching definition?	A definition reaching a point without being killed.	Path, no redefinition	Saying must reach
Direction?	Forward.	Definitions flow forward	Backward answer
May or must?	May analysis.	Union at joins	Saying intersection
What kills a definition?	Another definition of same variable.	Redefinition	Any statement
What is gen set?	Definitions created by a block that reach block end.	Block facts	Including killed defs
What is kill set?	Other definitions of variables redefined in block.	Same variable	Killing different variables
Equation?	OUT = GEN union (IN - KILL).	Standard equation	Wrong direction
Use in optimization?	Constant/copy propagation and use-def chains.	Applications	Only warnings
At branch joins?	Union definitions from predecessors.	May	Using intersection
Why ask it?	It is a standard data-flow example.	Fundamentals	Memorization only

7. Deep-Dive Questions

Question	Answer
How does SSA reduce reaching-definition complexity?	Each SSA variable has one definition, so use-def links are explicit.

Question	Answer
How are arrays handled?	Array element assignments may require alias/index analysis; compilers may approximate.
Why is reaching definitions conservative?	It includes any definition that may reach to avoid unsafe assumptions.
How does it help constant propagation?	If all reaching definitions assign the same constant, the use may be replaced.
What about function calls?	Calls may define globals or memory indirectly, depending on alias and side-effect analysis.

8. Comparison Tables

Reaching Definitions	Live-Variable Analysis
Forward analysis	Backward analysis
Tracks possible previous definitions	Tracks possible future uses
May analysis	May analysis
Helps propagation	Helps dead-code elimination

9. Common Mistakes

- Using intersection instead of union.
- Forgetting that redefinition kills old definitions.
- Treating all definitions as reaching forever.
- Ignoring branches.

10. Edge Cases / Special Cases

- Multiple definitions in one block: only the last definition of the same variable may be generated.
- Pointer writes can kill uncertain memory definitions.
- Uninitialized variables require special entry facts.
- Exception paths can add extra reaching possibilities.

11. How to Explain in Interview

"A reaching definition is an assignment that may still be the source of a variable's value at a later point because no redefinition blocks it along that path."

12. Quick Revision Notes

- Key definition: definition reaches if not killed on a path.
- Important equation: $OUT = GEN \cup (IN - KILL)$.
- Common comparison: reaching definitions vs liveness.
- Trap: it is forward may analysis.

13. Practice Tasks

1. Label definitions in a small program.
2. Compute GEN/KILL for each basic block.
3. Iterate IN/OUT to fixed point.
4. Use reaching definitions to find possible values at a use.

14. Final Cheat Sheet

Item	Answer
Core definition	Definitions that may reach a program point
Why it matters	Identifies possible sources of variable values
Most asked question	Equation and may/forward nature
Common comparison	Reaching definitions vs live variables
One-line answer	"It tells which assignments may still define a variable at a given point."

13. Live-Variable Analysis

1. Overview

Definition: A variable is live at a program point if its current value may be used in the future before being overwritten.

Why it matters: It enables dead-code elimination and register allocation.

Where used: Compilers, static analyzers, IDE warnings, bytecode optimizers.

Why interviewers ask: It is the classic backward data-flow analysis.

2. Core Idea

Intuition: Work backward and ask: "Will this value be needed later?"

Analogy: Packing for an exam: keep a pen if you will use it later; discard it if it will never be used.

Example:

```
x = 5;  
x = 6;  
print(x);
```

After `x = 5`, the value `5` is not live because it is overwritten before any use.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Use set	Variables read in block before definition	Makes variables live	$y=x+1$ uses x	USE/DEF
Def set	Variables assigned in block	Kills old values	$x=5$	Kill liveness
Backward direction	Facts flow from future to past	Natural for future use	exit to entry	Direction
May analysis	Variable may be used later	Union at branches	either branch uses x	Meet
Register allocation	Live ranges cannot share registers	Practical compiler backend	interference graph	Application

4. Real-World Example

Register allocators use live-variable analysis to decide which variables need registers at the same time. If two variables are not live simultaneously, they may share one register.

5. Diagrams / Mental Models

```

future use of x
  ^
  |
x is live backward until a definition overwrites it

```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is a live variable?	Current value may be used later before overwrite.	Future use	Variable exists in scope
Direction?	Backward.	From exits to entries	Forward answer
May or must?	May analysis.	Union from successors	Saying must
Equation?	$IN = USE \text{ union } (OUT - DEF)$.	Standard equation	Wrong gen/kill
How helps DCE?	Assignment to non-live variable may be removed if no side effects.	Dead store	Removing side-effect code
How helps registers?	Interfering live ranges need different registers.	Register allocation	Only DCE

Question	Clear Answer	Expected Points	Common Mistake
Is scope same as liveness?	No, scope is language visibility; liveness is value future-use.	Distinction	Equating both
What kills liveness?	A definition before any use.	DEF	A read
What happens at branches?	Live if used on any successor path.	Union	Intersection
Why ask it?	It tests backward reasoning and DCE.	Fundamentals	Memorized definition only

7. Deep-Dive Questions

Question	Answer
What is an interference graph?	Graph where variables live at same time are connected and cannot share a register.
Why is liveness a may analysis?	If a value may be used on any future path, it must be preserved.
How do function calls affect liveness?	Arguments are live before the call; caller-saved registers may be clobbered.
How does SSA help liveness?	SSA value uses define precise live ranges, though phi nodes need special handling.
Can a variable be live after function return?	Usually no local value is live after exit unless returned or stored observably.

8. Comparison Tables

Live-Variable Analysis	Available Expressions
Backward analysis	Forward analysis
May analysis	Must analysis
Tracks future variable uses	Tracks expressions already computed on all paths
Used for DCE/register allocation	Used for CSE

9. Common Mistakes

- Confusing variable scope with liveness.
- Using forward direction.
- Forgetting branch union.
- Removing assignments that write to volatile memory.

10. Edge Cases / Special Cases

- Phi nodes in SSA require edge-sensitive handling.
- Calls can use arguments and clobber registers.

- Volatile stores are observable even if value seems dead.
- Exception paths may use variables in handlers.

11. How to Explain in Interview

"A variable is live if its current value may be read later before being overwritten. Live-variable analysis works backward and is used for dead-code elimination and register allocation."

12. Quick Revision Notes

- Key definition: value may be used in future.
- Important equation: $IN = USE \cup (OUT - DEF)$.
- Common comparison: liveness vs reaching definitions.
- Trap: scope is not liveness.

13. Practice Tasks

1. Compute USE/DEF for each block.
2. Solve live-variable sets backward.
3. Mark dead assignments.
4. Build a small interference graph.

14. Final Cheat Sheet

Item	Answer
Core definition	Variable value may be used later
Why it matters	Enables DCE and register allocation
Most asked question	Equation and backward nature
Common comparison	Liveness vs reaching definitions
One-line answer	"A value is live if the program may still need it."

14. Available Expressions

1. Overview

Definition: An expression is available at a program point if it has already been computed on every path to that point and none of its operands have changed since.

Why it matters: It supports common subexpression elimination.

Where used: Optimizing compilers, query optimizers, JITs, and static analyzers.

Why interviewers ask: It is a standard forward must data-flow analysis.

2. Core Idea

Intuition: Reuse an expression only if all possible paths have already computed the same still-valid expression.

Analogy: A meeting note is "available" only if every team coming into the final meeting has already received the same note and nobody revised it.

Example:

```
if (c) t = a + b;
else t = a + b;
y = a + b;
```

$a + b$ is available before y if neither a nor b changed after both branches.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Expression generation	Block computes expression	Adds availability	$a+b$	GEN
Expression killing	Operand changes	Removes availability	$a=5$ kills $a+b$	KILL
Must analysis	Available on all paths	Safe reuse	intersection	Meet
Forward direction	Computations flow forward	Natural for previous computation	entry to exit	Direction
CSE use	Replaces repeated expression	Practical optimization	reuse temp	Application

4. Real-World Example

A database engine computing the same deterministic expression in multiple plan nodes can reuse it if it is available on all paths and no input column changed.

5. Diagrams / Mental Models

```
Path 1 computes a+b ----\
                          > join: available only if all paths computed it
Path 2 computes a+b ----/
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is an available expression?	Computed on all paths and operands unchanged.	All paths, no kill	Saying any previous computation
Direction?	Forward.	Computations flow forward	Backward
May or must?	Must analysis.	Intersection at joins	Union
What kills an expression?	Assignment to one of its operands.	Operand change	Killing unrelated expressions
Use in optimization?	CSE.	Reuse computation	Saying DCE mainly
Equation style?	$OUT = GEN \cup (IN - KILL)$, with intersection for IN.	Similar transfer, must meet	Wrong meet
Why intersection?	Expression must be available from every predecessor.	Safety	Using union
Can memory expressions be available?	Only if memory not modified.	Alias analysis	Always yes
How differs from reaching definitions?	Available expressions is must; reaching definitions is may.	Analysis type	Mixing
Why ask it?	It tests must data-flow and CSE.	Fundamentals	Definition only

7. Deep-Dive Questions

Question	Answer
Why initialize available expressions differently?	Must analyses often start with universal set except entry constraints.
What is anticipated expression analysis?	A backward analysis for expressions that will be used on all future paths.
How does partial redundancy elimination relate?	It uses availability/anticipation ideas to remove computations repeated on some paths.
Why are memory expressions hard?	Stores and aliases may change loaded values.
How do exceptions affect availability?	Exceptional control-flow paths can break "all paths" assumptions.

8. Comparison Tables

Available Expressions	Reaching Definitions
Forward must analysis	Forward may analysis

Available Expressions	Reaching Definitions
Uses intersection at joins	Uses union at joins
Supports CSE	Supports propagation
Killed by operand modification	Killed by same-variable redefinition

9. Common Mistakes

- Using union instead of intersection.
- Ignoring operand redefinition.
- Calling expressions available if only one branch computes them.
- Forgetting memory and side effects.

10. Edge Cases / Special Cases

- Expressions involving volatile loads are not safely available.
- Floating-point expressions may require exact semantic constraints.
- Function calls may kill memory-based expressions.
- Entry block usually starts with no available expressions.

11. How to Explain in Interview

"An expression is available if every path to a point has already computed it and no operand has changed, so the compiler can safely reuse the result."

12. Quick Revision Notes

- Key definition: computed on all paths and not killed.
- Important point: forward must analysis.
- Common comparison: available expressions vs reaching definitions.
- Trap: join uses intersection.

13. Practice Tasks

1. Compute available expressions for an `if-else`.
2. Identify expression kill sets.
3. Use available expressions to perform CSE.
4. Explain why one-path computation is not enough.

14. Final Cheat Sheet

Item	Answer
Core definition	Expression already computed on all paths
Why it matters	Enables safe CSE
Most asked question	Why is it must analysis?
Common comparison	Available expressions vs reaching definitions

Item	Answer
One-line answer	"Available means already computed on every path and still valid."

15. Control-Flow Optimization

1. Overview

Definition: Control-flow optimization simplifies branches, jumps, and basic-block structure while preserving behavior.

Why it matters: Cleaner control flow improves branch prediction, reduces instruction count, and enables other optimizations.

Where used: Compiler middle ends, bytecode optimizers, JIT compilers, database query plan optimizers.

Why interviewers ask: It tests CFG reasoning and branch simplification.

2. Core Idea

Intuition: Remove unnecessary paths and make execution flow simpler.

Analogy: If a navigation route says "turn left, then immediately U-turn back," simplify the route.

Example:

```
if (true) {
  x = 1;
} else {
  x = 2;
}
```

becomes:

```
x = 1;
```

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Branch folding	Replace known branch	Removes unreachable code	<code>if(false)</code>	Constant folding link
Jump threading	Bypass intermediate jumps	Shorter paths	jump to jump	CFG cleanup

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Block merging	Combine blocks	Reduces CFG complexity	single predecessor/successor	Basic blocks
Unreachable removal	Delete dead blocks	Smaller code	after return	Reachability
Tail duplication	Duplicate small blocks to simplify branches	May improve prediction	small join block	Trade-off

4. Real-World Example

A JavaScript JIT may specialize a hot function where a condition is always true for observed types. It can simplify control flow in the optimized version and deoptimize if assumptions fail.

5. Diagrams / Mental Models

Before:

```
B1 -> B2 -> B4
  \-> B3 -> B4
```

After known condition:

```
B1 -> B2 -> B4
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is control-flow optimization?	Simplifying branches/jumps/CFG.	CFG focus	Talking only arithmetic
Give examples.	Branch folding, jump threading, block merging.	Multiple forms	Only DCE
How does constant propagation help?	It may make branch conditions known.	Pass interaction	Treating isolated
What is jump threading?	Redirecting control through known branch paths.	Bypass jumps	Confusing with threads
What is block merging?	Combining adjacent blocks when safe.	CFG simplification	Merging arbitrary blocks
What is unreachable block removal?	Delete blocks with no path from entry.	Reachability	Removing rarely executed code
Does it affect branch prediction?	Simpler hot paths can help.	CPU behavior	Saying only size

Question	Clear Answer	Expected Points	Common Mistake
Can it increase code size?	Some forms like tail duplication can.	Trade-off	Assuming always smaller
Is it local or global?	Usually CFG-level global within function.	Scope	Calling peephole only
Why ask it?	It checks CFG understanding.	Branches and blocks	Vague speed answer

7. Deep-Dive Questions

Question	Answer
How does CFG simplification enable later passes?	Simpler blocks expose constants, dead code, and easier dominance relationships.
What is tail duplication?	Copying a small shared successor into predecessors to remove jumps or improve layout.
What is branch inversion?	Reversing a condition to improve fall-through layout.
How do exception edges complicate CFG optimization?	Blocks may have hidden exceptional successors that cannot be ignored.
What is deoptimization in JITs?	Returning to less optimized code if speculative control-flow assumptions fail.

8. Comparison Tables

Control-Flow Optimization	Dead-Code Elimination
Simplifies CFG structure	Removes useless instructions/blocks
Focuses on branches and jumps	Focuses on observable effect
Example: merge blocks	Example: remove dead store
Often enables DCE	Often follows CFG simplification

9. Common Mistakes

- Confusing jump threading with OS threads.
- Removing rarely executed code as if it were unreachable.
- Ignoring exception edges.
- Assuming all control-flow simplification reduces code size.

10. Edge Cases / Special Cases

- Switch statements need careful range and default handling.
- Exception handling creates extra control-flow edges.

- Branch prediction metadata can guide layout.
- Tail duplication trades code size for speed.

11. How to Explain in Interview

"Control-flow optimization simplifies the program's CFG by removing impossible branches, redirecting jumps, merging blocks, and deleting unreachable paths."

12. Quick Revision Notes

- Key definition: optimize branches and CFG.
- Important point: distinguishes unreachable from cold code.
- Common comparison: CFG optimization vs DCE.
- Trap: exception edges.

13. Practice Tasks

1. Draw CFG before and after branch folding.
2. Remove unreachable blocks from a CFG.
3. Perform jump threading on a small example.
4. Merge basic blocks with single predecessor/successor.

14. Final Cheat Sheet

Item	Answer
Core definition	Simplify control-flow graph
Why it matters	Fewer branches, better optimization opportunities
Most asked question	Difference between unreachable and rarely executed code
Common comparison	Control-flow optimization vs DCE
One-line answer	"It makes the CFG simpler without changing which observable behavior occurs."

16. Inlining

1. Overview

Definition: Inlining replaces a function call with the body of the called function.

Why it matters: It removes call overhead and exposes caller/callee code to further optimization.

Where used: C++ compilers, JVM JIT, JavaScript engines, Rust/Go compilers, database UDF optimization.

Why interviewers ask: It tests trade-offs between speed, code size, and optimization exposure.

2. Core Idea

Intuition: Instead of jumping to a small function, paste its body at the call site.

Analogy: Instead of saying "see appendix A" for a two-line explanation, write those two lines directly.

Example:

```
int square(int x) { return x * x; }  
y = square(a + 1);
```

After inlining:

```
int t = a + 1;  
y = t * t;
```

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Call overhead	Cost of call/return	Removed by inlining	stack frame	Performance
Optimization exposure	Enables propagation/CSE	Main benefit	inline then fold	Bigger than call saving
Code size	Body duplicated	Can hurt cache	large function	Trade-off
Recursive inlining	Inlining recursive calls	Needs limits	factorial	Termination
Virtual call inlining	Inline after devirtualization	Powerful in OOP/JIT	Java methods	Runtime type info

4. Real-World Example

Java JIT compilers inline small frequently called methods like getters. After inlining, the JIT may eliminate object allocations or fold constants.

5. Diagrams / Mental Models

```
Before:  
caller -> call f -> return  
  
After:  
caller contains f's body directly
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is inlining?	Replacing a call with callee body.	Function body at call site	Only macro substitution
Why useful?	Removes call overhead and enables more optimization.	Exposure	Only call overhead
Downside?	Code size growth and instruction-cache pressure.	Trade-off	Saying always good
Is <code>inline</code> keyword a command?	Usually a hint/request; compiler decides.	Language nuance	Assuming guaranteed
Can recursive functions be inlined?	Partially, with limits.	Recursion control	Infinite expansion
What is devirtualization?	Resolving dynamic call target so it can be inlined.	OOP/JIT	Confusing with overriding
Does inlining preserve semantics?	It must, including exceptions and evaluation order.	Correctness	Macro-like unsafe answer
Why can inlining enable DCE?	Unused callee work may become visible and removable.	Pass interaction	Isolated view
How does profile help?	Hot call sites are better inline candidates.	PGO	Inline all small functions
Why ask it?	It tests optimization trade-offs.	Speed vs size	"Faster always"

7. Deep-Dive Questions

Question	Answer
Why is inlining often more valuable than call overhead removal?	It exposes constants, aliases, and object allocation patterns across the call boundary.
What is an inline budget?	Compiler limit controlling code-size growth from inlining.
How does JIT inlining differ from ahead-of-time inlining?	JIT uses runtime type/profile information and can deoptimize.
Why can inlining hurt performance?	Larger code may reduce instruction-cache locality and increase compile time.
What is always-inline?	A compiler attribute requesting aggressive inlining, still subject to semantic constraints.

8. Comparison Tables

Inlining	Macro Expansion
Compiler optimization on functions	Preprocessor/text substitution

Inlining	Macro Expansion
Type-checked function semantics	Can have textual side effects
Compiler may choose	Macro always expands before compilation
Preserves function behavior	Can duplicate argument side effects

9. Common Mistakes

- Thinking `inline` guarantees inlining.
- Saying only call overhead matters.
- Ignoring code bloat.
- Confusing inlining with macros.

10. Edge Cases / Special Cases

- Large functions may not be profitable.
- Recursive functions need depth limits.
- Virtual calls require target resolution.
- Debug stack traces can become less direct.

11. How to Explain in Interview

"Inlining replaces a function call with the function body at the call site, mainly to expose more optimization opportunities, but it can increase code size."

12. Quick Revision Notes

- Key definition: call replaced by body.
- Important point: exposure often matters more than call overhead.
- Common comparison: inlining vs macro expansion.
- Trap: inline keyword is not always a guarantee.

13. Practice Tasks

1. Manually inline a small function.
2. Show constant propagation after inlining.
3. Explain why inlining a large function may hurt.
4. Inspect compiler output for a small getter.

14. Final Cheat Sheet

Item	Answer
Core definition	Replace call with callee body
Why it matters	Removes call overhead and exposes optimization
Most asked question	Why can inlining hurt?
Common comparison	Inlining vs macros

Item	Answer
One-line answer	"Inlining pastes a function body at the call site when it is profitable and safe."

17. Interprocedural Optimization

1. Overview

Definition: Interprocedural optimization, or IPO, optimizes across function boundaries instead of treating each function independently.

Why it matters: Many performance and correctness facts are hidden behind function calls.

Where used: Link-time optimization, whole-program optimization, JVM/JIT compilers, C++ compilers, static analyzers.

Why interviewers ask: It tests your understanding beyond single-function CFGs.

2. Core Idea

Intuition: Understand how functions call each other and share data, then optimize with cross-function knowledge.

Analogy: Managing one department locally is useful, but company-wide optimization needs knowing how departments interact.

Example:

```
int add5(int x) { return x + 5; }  
int y = add5(10);
```

IPO can inline `add5`, propagate `10`, and fold to `15`.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Call graph	Graph of function calls	Backbone of IPO	<code>main -> f -> g</code>	Build graph
Inlining	Replace calls	Common IPO action	small function	Trade-off
Escape analysis	Determine if object escapes	Enables stack allocation	local object	Memory optimization
Devirtualization	Resolve dynamic dispatch	Enables inlining	virtual method	OOP/JIT

Subtopic	Meaning	Why It Matters	Example	Interview Angle
LTO	Optimize at link time	Sees multiple files	C++ link-time optimization	Build systems
Function specialization	Clone function for known arguments	Enables constants	<code>f(true)</code>	Code size tradeoff

4. Real-World Example

In C++ link-time optimization, the compiler can see functions from multiple source files. It may inline a small function defined in another file and remove unused functions from the final binary.

5. Diagrams / Mental Models

```

main
| \
v  v
f  g
|
v
h

```

IPO uses this call graph to optimize across calls.

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is IPO?	Optimization across function boundaries.	Call graph	Saying global optimization only
Difference from intraprocedural?	Intra is within one function; inter crosses functions.	Scope	Confusing terms
What is call graph?	Graph showing which functions call which.	IPO structure	CFG answer
Give IPO examples.	Inlining, devirtualization, escape analysis, specialization.	Multiple examples	Only inlining
What is LTO?	Link-time optimization across compiled units.	Whole-program view	Runtime linking
Why is IPO hard?	Dynamic dispatch, pointers, separate compilation, recursion.	Precision issues	"More code" only

Question	Clear Answer	Expected Points	Common Mistake
What is escape analysis?	Determines whether object/reference leaves scope.	Stack allocation	Memory leak analysis only
What is devirtualization?	Resolving virtual call target statically/dynamically.	OOP optimization	Removing virtual keyword
Can IPO increase code size?	Yes, inlining/specialization duplicate code.	Trade-off	Always smaller
Why ask it?	It tests advanced compiler scope.	Beyond CFG	Vague answer

7. Deep-Dive Questions

Question	Answer
How does separate compilation limit IPO?	A compiler may not see bodies from other translation units unless using LTO.
How does recursion affect call graphs?	It creates cycles that require careful fixed-point or limited transformations.
What is context-sensitive analysis?	It distinguishes facts for different call sites instead of merging all calls.
Why does dynamic loading complicate IPO?	New code may override or call functions unknown at compile time.
How does JIT IPO differ?	JIT can use runtime profiles and speculative assumptions with deoptimization.

8. Comparison Tables

Intraprocedural Optimization	Interprocedural Optimization
Within one function	Across functions
Uses CFG	Uses CFG plus call graph
Simpler and cheaper	More powerful and expensive
Example: LICM	Example: cross-function inlining

9. Common Mistakes

- Treating global optimization and IPO as identical.
- Assuming every call target is statically known.
- Ignoring separate compilation.
- Forgetting code-size growth from specialization.

10. Edge Cases / Special Cases

- Function pointers and reflection obscure call targets.
- Dynamic libraries can restrict whole-program assumptions.
- Recursion creates call-graph cycles.
- External visibility rules may prevent removing functions.

11. How to Explain in Interview

"Interprocedural optimization analyzes multiple functions together using a call graph, enabling optimizations like cross-function inlining, specialization, and escape analysis."

12. Quick Revision Notes

- Key definition: optimize across function calls.
- Important point: call graph is central.
- Common comparison: intra vs interprocedural.
- Trap: dynamic dispatch and separate compilation.

13. Practice Tasks

1. Draw a call graph for a small program.
2. Inline a small cross-function call and fold constants.
3. Identify whether an object escapes a function.
4. Explain how LTO helps C++ optimization.

14. Final Cheat Sheet

Item	Answer
Core definition	Cross-function optimization
Why it matters	Unlocks facts hidden behind calls
Most asked question	Difference from intraprocedural optimization
Common comparison	IPO vs local/global optimization
One-line answer	"IPO optimizes with knowledge of how functions call and affect each other."

18. Profile-Guided Optimization

1. Overview

Definition: Profile-guided optimization, or PGO, uses runtime execution data to guide compiler optimization decisions.

Why it matters: Real programs have hot and cold paths; PGO helps optimize what actually runs often.

Where used: Browsers, operating systems, game engines, databases, backend services, large C++/Rust/Go applications.

Why interviewers ask: It tests whether you understand that optimization decisions depend on runtime behavior.

2. Core Idea

Intuition: Measure first, then optimize based on real execution frequency.

Analogy: A city improves roads after traffic surveys, not by guessing which roads are busy.

Example workflow:

1. Build instrumented binary
2. Run representative workload
3. Collect profile data
4. Rebuild optimized binary using profile

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Instrumentation	Add counters to collect data	Accurate profiles	branch counters	Build workflow
Sampling	Periodically observe execution	Lower overhead	perf samples	Accuracy trade-off
Hot/cold code	Frequently/rarely executed paths	Optimize hot, shrink cold	error path cold	Branch layout
Branch probabilities	Likely branch direction	Improves layout/prediction	<code>if likely</code>	CPU pipeline
Profile representativeness	Workload matches production	Avoids bad optimization	realistic tests	PGO risk

4. Real-World Example

Browser teams use PGO to optimize startup and page-loading paths. Hot functions are laid out close together, likely branches become fall-through paths, and cold error handling may be moved away.

5. Diagrams / Mental Models

```
Run program -> collect counts -> rebuild -> optimize hot paths
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is PGO?	Optimization guided by runtime profile data.	Measure then optimize	Static optimization only
What data is collected?	Function counts, branch counts, value/type profiles.	Profile types	Only time measurement
Why useful?	Optimizes hot paths and improves branch/layout decisions.	Real workload	Generic faster answer
Workflow?	Instrument, run workload, collect, rebuild.	Steps	Missing rebuild
What is hot code?	Code executed frequently.	Frequency	Code that is important logically
What is cold code?	Rarely executed code, often error paths.	Layout/size	Dead code
Can PGO hurt?	Yes, if profile is unrepresentative.	Risk	Always beneficial
How helps inlining?	Inline hot call sites more aggressively.	Cost model	Inline all functions
Sampling vs instrumentation?	Sampling lower overhead; instrumentation more precise.	Trade-off	Treating same
Why ask it?	It tests real-world optimization awareness.	Runtime data	Only textbook passes

7. Deep-Dive Questions

Question	Answer
What is value profiling?	Recording common runtime values to specialize code.
How does PGO improve code layout?	It places hot blocks together and makes likely branches fall through.
Why is representative workload critical?	Optimizing for the wrong workload can slow real production behavior.
What is AutoFDO?	A sampling-based PGO approach using profiles from production-like runs.
How does PGO interact with devirtualization?	Type profiles can reveal likely targets for virtual calls.

8. Comparison Tables

Static Optimization	Profile-Guided Optimization
Uses compile-time analysis only	Uses runtime execution data
Conservative without frequency	Knows hot/cold paths
No training run needed	Needs representative profiling
Less workload-specific	More workload-specific

9. Common Mistakes

- Thinking PGO is the same as benchmarking.
- Using unrealistic training inputs.
- Assuming cold code is dead code.
- Forgetting the second optimized build.

10. Edge Cases / Special Cases

- Profiles can become stale after code changes.
- Security-sensitive code may avoid certain speculation.
- Multi-tenant workloads may have conflicting hot paths.
- Instrumentation overhead can distort behavior.

11. How to Explain in Interview

"PGO runs the program on representative workloads, records which paths are hot, and recompiles using that data to make better inlining, layout, and branch decisions."

12. Quick Revision Notes

- Key definition: runtime-profile-guided compilation.
- Important point: profile quality matters.
- Common comparison: static optimization vs PGO.
- Trap: cold code is not dead code.

13. Practice Tasks

1. Describe a PGO build pipeline.
2. Mark hot and cold paths in sample code.
3. Explain how branch probabilities change layout.
4. Compare representative vs poor training data.

14. Final Cheat Sheet

Item	Answer
Core definition	Use runtime profiles to guide optimization
Why it matters	Optimizes actual hot paths
Most asked question	Can PGO hurt?

Item	Answer
Common comparison	Static optimization vs PGO
One-line answer	"PGO lets the compiler optimize based on how the program really runs."

19. Auto-Vectorization

1. Overview

Definition: Auto-vectorization is when the compiler automatically converts scalar operations into SIMD vector operations.

Why it matters: SIMD instructions process multiple data elements per instruction, improving throughput for loops over arrays.

Where used: Scientific computing, ML kernels, image/video processing, databases, compression, cryptography, game engines.

Why interviewers ask: It tests knowledge of hardware-aware optimization, loops, dependencies, and memory layout.

2. Core Idea

Intuition: If loop iterations are independent, process several elements at once using vector registers.

Analogy: Instead of stamping one paper at a time, use a stamp machine that marks eight papers in one press.

Example:

```
for (int i = 0; i < n; i++) {  
    c[i] = a[i] + b[i];  
}
```

SIMD idea:

```
load vector a[i..i+3]  
load vector b[i..i+3]  
add vectors  
store vector c[i..i+3]
```

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
SIMD	Single instruction, multiple data	Hardware basis	AVX/NEON	CPU architecture

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Dependence analysis	Check iterations independent	Correctness	<code>a[i]=a[i-1]+1</code> blocks	Loop-carried dependencies
Alignment	Memory address vector-friendly	Performance	32-byte aligned loads	Memory layout
Remainder handling	Leftover elements	Correctness	<code>n % vector_width</code>	Cleanup loop
Reductions	Combine many values	Harder vectorization	sum array	Associativity
Aliasing	Pointers may overlap	Can block vectorization	<code>c</code> aliases <code>a</code>	<code>restrict</code>

4. Real-World Example

A database engine scanning a column can compare many values at once using SIMD:

```
vector_load prices
vector_compare prices > 100
produce mask of matching rows
```

This is common in analytical databases.

5. Diagrams / Mental Models

```
Scalar:
i=0: c0=a0+b0
i=1: c1=a1+b1
i=2: c2=a2+b2
i=3: c3=a3+b3

Vector:
[c0 c1 c2 c3] = [a0 a1 a2 a3] + [b0 b1 b2 b3]
```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
What is auto-vectorization?	Compiler converts scalar loop to SIMD.	Automatic SIMD	Manual threading
What is SIMD?	One instruction operates on multiple data lanes.	Vector lanes	Multiple CPU cores

Question	Clear Answer	Expected Points	Common Mistake
When can a loop be vectorized?	Iterations are independent and memory access is suitable.	Dependence	Any loop
What blocks vectorization?	Loop-carried dependencies, aliasing, complex control flow.	Safety	Only data type
What is remainder handling?	Scalar/vector cleanup for leftover elements.	Correctness	Ignoring <code>n</code> not divisible
Why does alignment matter?	Aligned loads/stores may be faster or required on some targets.	Memory	Syntax issue
How is it different from unrolling?	Vectorization uses SIMD hardware; unrolling duplicates scalar body.	Hardware vs code layout	Treating same
What are reductions?	Operations like sum/min over many elements.	Special vector patterns	Saying impossible always
How can <code>restrict</code> help?	It tells compiler pointers do not alias.	Alias proof	Magic speed keyword
Why ask it?	It connects compiler optimization to CPU architecture.	Hardware-aware thinking	Theoretical answer only

7. Deep-Dive Questions

Question	Answer
Why does <code>a[i] = a[i-1] + 1</code> block vectorization?	Each iteration depends on the previous iteration's result.
How are reductions vectorized?	Partial sums are computed in vector lanes, then horizontally combined.
What is vector width?	Number of elements processed per SIMD instruction, depending on type and ISA.
How do masks help vectorization?	Masked operations handle conditions or tails without scalar branches.
Why can floating-point vectorization change results?	Reordering operations can change rounding due to non-associativity.

8. Comparison Tables

Auto-Vectorization	Multithreading
Uses SIMD lanes in one core	Uses multiple threads/cores

Auto-Vectorization	Multithreading
Best for data-parallel loops	Best for larger independent tasks
Compiler may do automatically	Usually programmer/runtime controlled
Limited by dependencies and memory layout	Limited by synchronization and scheduling

Auto-Vectorization	Loop Unrolling
Uses vector instructions	Duplicates scalar instructions
Processes lanes simultaneously	Reduces loop overhead
Hardware-dependent	Mostly compiler transformation
Requires dependence proof	Requires trip-count/remainder handling

9. Common Mistakes

- Confusing SIMD with multithreading.
- Ignoring loop-carried dependencies.
- Forgetting aliasing.
- Assuming vectorization always preserves floating-point bitwise results.

10. Edge Cases / Special Cases

- Small loops may not benefit due to setup overhead.
- Non-contiguous memory access may require gather/scatter instructions.
- Conditional loops may need masks.
- Floating-point reductions may need relaxed math flags.

11. How to Explain in Interview

"Auto-vectorization is when the compiler transforms independent scalar loop iterations into SIMD instructions so multiple array elements are processed per CPU instruction."

12. Quick Revision Notes

- Key definition: scalar loop to SIMD.
- Important point: independence and alias analysis.
- Common comparison: vectorization vs multithreading.
- Trap: SIMD is not multiple threads.

13. Practice Tasks

1. Identify whether five loops are vectorizable.
2. Rewrite a loop to use contiguous arrays.
3. Explain why pointer aliasing blocks vectorization.
4. Compile with vectorization reports and inspect messages.

14. Final Cheat Sheet

Item	Answer
Core definition	Automatic conversion to SIMD operations
Why it matters	Processes multiple data elements per instruction
Most asked question	What prevents vectorization?
Common comparison	SIMD vs multithreading
One-line answer	"Auto-vectorization lets the compiler use vector hardware for independent loop iterations."

20. Optimization Pass Interactions

This section connects the listed topics because interviewers often ask how optimizations work together instead of asking each pass in isolation.

1. Overview

Definition: Optimization pass interaction means one optimization exposes opportunities for another.

Why it matters: Real compilers run many passes repeatedly because a single pass rarely gives the final best code.

Where used: LLVM, GCC, HotSpot JVM, V8, database query optimizers, shader compilers.

Why interviewers ask: It tests whether you understand compiler pipelines, not just isolated definitions.

2. Core Idea

Intuition: Optimizations form a chain reaction.

Small example:

```
int x = 10;
int y = x + 5;
if (y > 20) print(y);
else print(0);
```

Steps:

1. Constant propagation: `x -> 10`.
2. Constant folding: `y = 15`.
3. Branch folding: `15 > 20 -> false`.
4. Control-flow optimization: remove true branch.
5. DCE: remove unused assignments if no longer needed.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Pass ordering	Order affects opportunities	Better results	propagate before DCE	Pipeline reasoning
Fixed-point pipelines	Repeat until no change	Exposes chained changes	fold, DCE, fold again	Iterative optimization
Profitability	Correct does not mean worthwhile	Avoid slow compile/bloat	inlining too much	Cost model
Safety	Meaning must not change	Core compiler rule	volatile access	Semantic preservation
Debuggability	Optimized code differs from source	Practical concern	missing variables	Real systems

4. Real-World Example

A JIT compiler may first inline a hot call, then propagate constants from the caller, eliminate branches inside the callee, remove dead stores, and finally generate vectorized machine code for the remaining loop.

5. Diagrams / Mental Models

```

Inlining
|
v
Constant propagation -> Constant folding
|           |
v           v
Branch simplification -> Dead-code elimination
|
v
Loop optimization -> Vectorization

```

6. Common Interview Questions

Question	Clear Answer	Expected Points	Common Mistake
Why run multiple passes?	One pass exposes opportunities for another.	Interaction	One-pass compiler view
Why does order matter?	Some passes need facts created by earlier passes.	Pipeline	Random order
Example chain?	Propagation -> folding -> DCE.	Concrete chain	Listing names only

Question	Clear Answer	Expected Points	Common Mistake
Can optimization hurt?	Yes, compile time, code size, cache, profile mismatch.	Trade-offs	Always faster
What is fixed point?	Repeat until no pass changes code.	Stabilization	Infinite pass idea
Why preserve semantics?	Optimized code must behave the same observably.	Correctness	"Faster is enough"
What is cost model?	Estimate whether transformation is profitable.	Profitability	Correct equals profitable
Why debug harder?	Variables/code may be removed or reordered.	Practical systems	No mention
How does inlining help propagation?	Caller constants become visible inside callee.	Cross-pass	Isolated inlining
How does PGO guide passes?	Hot paths get more aggressive optimization.	Runtime data	PGO as benchmark only

7. Deep-Dive Questions

Question	Answer
Why do compilers use IR?	IR makes analysis and transformations easier than raw source or machine code.
Why have multiple IR levels?	High-level IR preserves structure; low-level IR matches target details.
What is canonicalization?	Rewriting code into standard forms so later passes recognize patterns.
Why not run every optimization always?	Compile time, code size, and missed profitability can hurt.
How does deoptimization support aggressive JIT optimization?	JIT can speculate and fall back if assumptions fail.

8. Comparison Tables

Correctness	Profitability
Is transformation legal?	Is transformation worth doing?
Must always hold	Depends on target/workload
Based on semantics	Based on cost model/profile
Example: cannot remove side effect	Example: may skip unrolling due to code size

9. Common Mistakes

- Studying optimizations as isolated tricks.
- Forgetting pass ordering.
- Assuming correct transformations are always profitable.
- Ignoring debug and compile-time trade-offs.

10. Edge Cases / Special Cases

- Optimization for speed may conflict with optimization for size.
- Debug builds may disable aggressive transformations.
- Security hardening may block some optimizations.
- JIT compilers may optimize speculatively and deoptimize.

11. How to Explain in Interview

"Compiler optimizations interact: propagation can expose folding, folding can expose unreachable code, DCE can clean it up, and loop or vector passes can then optimize the remaining hot code."

12. Quick Revision Notes

- Key definition: passes expose opportunities for other passes.
- Important point: legal is not always profitable.
- Common comparison: correctness vs profitability.
- Trap: real compilers use repeated pipelines.

13. Practice Tasks

1. Optimize a small program step by step using three passes.
2. Explain why inlining before constant propagation helps.
3. Compare speed-focused and size-focused optimization choices.
4. Draw a simple optimization pipeline.

14. Final Cheat Sheet

Item	Answer
Core definition	Optimization passes help and depend on each other
Why it matters	Real compilers optimize through pipelines
Most asked question	Give a pass interaction example
Common comparison	Correctness vs profitability
One-line interview answer	"Optimizations are chained: each simplification can reveal the next one."

Master Revision Tables

Optimization Summary

Topic	Main Purpose	Typical Analysis Needed	Main Risk
Constant folding	Compile-time expression evaluation	Type/semantic checks	Overflow, float behavior
Constant propagation	Replace variables with constants	Reaching definitions/data-flow	Stale facts
Dead-code elimination	Remove useless code	Liveness/reachability	Removing side effects
CSE	Reuse repeated expressions	Available expressions/value numbering	Operand changes, aliasing
Copy propagation	Remove unnecessary copies	Reaching copies/use-def	Source/destination changes
Strength reduction	Cheaper equivalent operations	Induction variables/cost model	Incorrect arithmetic semantics
LICM	Hoist invariant loop work	Loop/dominance/alias analysis	Speculation and side effects
Loop unrolling	Reduce loop overhead	Trip count/cost model	Code bloat
Peephole optimization	Local instruction cleanup	Pattern matching/local facts	Machine flags/semantics
Local optimization	Optimize one basic block	Basic block scan	Limited scope
Global optimization	Optimize across CFG	Data-flow	Complexity
Data-flow analysis	Compute facts over CFG	IN/OUT, gen/kill	Wrong direction/meet
Reaching definitions	Find possible value sources	Forward may analysis	Using intersection
Live-variable analysis	Find values needed later	Backward may analysis	Confusing with scope
Available expressions	Find reusable expressions	Forward must analysis	Using union
Control-flow optimization	Simplify branches/CFG	CFG reachability	Ignoring exception edges
Inlining	Replace calls with bodies	Call graph/cost model	Code size
IPO	Optimize across functions	Call graph/alias/escape analysis	Dynamic dispatch

Topic	Main Purpose	Typical Analysis Needed	Main Risk
PGO	Use runtime profiles	Instrumentation/sampling	Bad training workload
Auto-vectorization	Use SIMD automatically	Dependence/alias analysis	Loop dependencies

Interview Must-Remember Facts

Concept	Must Remember
Semantic preservation	Optimized code must keep observable behavior
May analysis	Fact true on at least one path, usually union
Must analysis	Fact true on all paths, usually intersection
Forward analysis	Facts flow in execution direction
Backward analysis	Facts flow opposite execution direction
Basic block	Straight-line code with one entry and one exit
CFG	Graph of basic blocks and possible control transfers
Call graph	Graph of functions and call relationships
Cost model	Decides whether a legal optimization is profitable
Alias analysis	Determines whether references may access same memory

One-Minute Interview Answer

"Code optimization improves generated code without changing observable behavior. Simple passes include constant folding, constant propagation, copy propagation, CSE, and dead-code elimination. Loop optimizations include LICM, strength reduction, unrolling, and auto-vectorization. Data-flow analysis provides the facts needed for safe optimization, such as reaching definitions, live variables, and available expressions. Real compilers also use interprocedural optimization and profile-guided optimization to make better decisions across functions and hot runtime paths."